```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
Start coding or generate with AI.
```

```
import pandas as pd
```

```
df=pd.read_csv('/content/drive/MyDrive/Teen_Mental_Health.csv')
```

```
df
```

	age	gender	daily_social_media_hours	platform_usage	sleep_hours	screen_time_bef
0	14	male	7.9	Facebook	7.4	
1	19	female	1.9	TikTok	8.0	
2	17	female	1.3	Instagram	7.6	
3	15	male	7.4	YouTube	6.9	
4	15	female	4.7	All Platforms	4.9	
...	...	...	...	...	...	
1195	18	female	6.8	Instagram	6.6	
1196	16	male	2.3	All Platforms	8.0	
1197	14	female	1.7	All Platforms	8.7	
1198	15	male	3.9	All Platforms	8.5	
1199	16	female	4.7	TikTok	6.5	

1200 rows × 16 columns

```
df.isnull().sum()
```

	0
age	0
gender	0
daily_social_media_hours	0
platform_usage	0
sleep_hours	0
screen_time_before_sleep	0
academic_performance	0
physical_activity	0
social_interaction_level	0
stress_level	0
anxiety_level	0
addiction_level	0
depression_label	0
mental_health_risk_score	0
sleep_quality	0
digital_wellbeing_flag	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
```

```
le_gender=LabelEncoder()
df['gender']=le_gender.fit_transform(df['gender'])
```

```
le_platform=LabelEncoder()
df['platform_usage']=le_platform.fit_transform(df['platform_usage'])
```

```
le_social=LabelEncoder()
df['social_interaction_level']=le_social.fit_transform(df['social_interaction_level'])
```

```
le_sleep=LabelEncoder()
df['sleep_quality']=le_sleep.fit_transform(df['sleep_quality'])
```

```
le_digital=LabelEncoder()
df['digital_wellbeing_flag']=le_digital.fit_transform(df['digital_wellbeing_flag'])
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df['gender'] = le.fit_transform(df['gender'])
df['platform_usage'] = le.fit_transform(df['platform_usage'])
df['social_interaction_level'] = le.fit_transform(df['social_interaction_level'])
df['sleep_quality'] = le.fit_transform(df['sleep_quality'])
```

```
df['digital_wellbeing_flag'] = le.fit_transform(df['digital_wellbeing_flag'])

df[['gender', 'platform_usage', 'social_interaction_level', 'sleep_quality', 'digital_w
```

	gender	platform_usage	social_interaction_level	sleep_quality	digital_wellbeing_fla
0	1	1	1	0	
1	0	3	0	1	
2	0	2	0	0	
3	1	4	2	0	
4	0	0	2	2	

```
from sklearn.preprocessing import StandardScaler

# Calculate the median of the original mental_health_risk_score to use as a dynamic thr
median_risk_score = df['mental_health_risk_score'].median()
print(f"Median mental_health_risk_score used for binarization: {median_risk_score}")

df['mental_health_risk_score'] = df['mental_health_risk_score'].apply(
    lambda x: 0 if x <= median_risk_score else 1
)

# Print value counts to confirm the new distribution of the target variable
print("Value counts for the binarized target variable 'mental_health_risk_score':")
print(df['mental_health_risk_score'].value_counts())

X = df.drop(columns=['mental_health_risk_score'])
y = df['mental_health_risk_score']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
Median mental_health_risk_score used for binarization: 17.0
Value counts for the binarized target variable 'mental_health_risk_score':
mental_health_risk_score
0    667
1    533
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, y, test_size=0.2, random_
```

```
from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train,Y_train)
Y_pred=knn.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
accuracy=accuracy_score(Y_test,Y_pred)
print(f"Model Accuracy:{accuracy*100:.2f}%")
print("\n---Classification Report---")
print(classification_report(Y_test,Y_pred))
```

```
Model Accuracy:83.75%

---Classification Report---
```

```
              precision    recall  f1-score   support

           0       0.80      0.90      0.85       122
           1       0.88      0.77      0.82       118

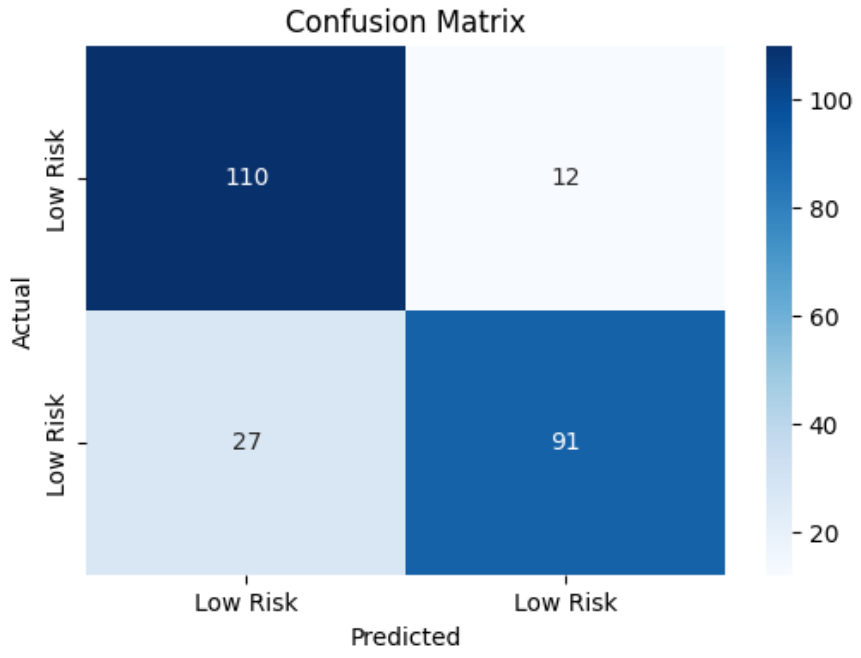
    accuracy                           0.84       240
   macro avg       0.84      0.84      0.84       240
weighted avg       0.84      0.84      0.84       240
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(Y_test, Y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Low Risk', 'Low Risk'],
            yticklabels=['Low Risk', 'Low Risk'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



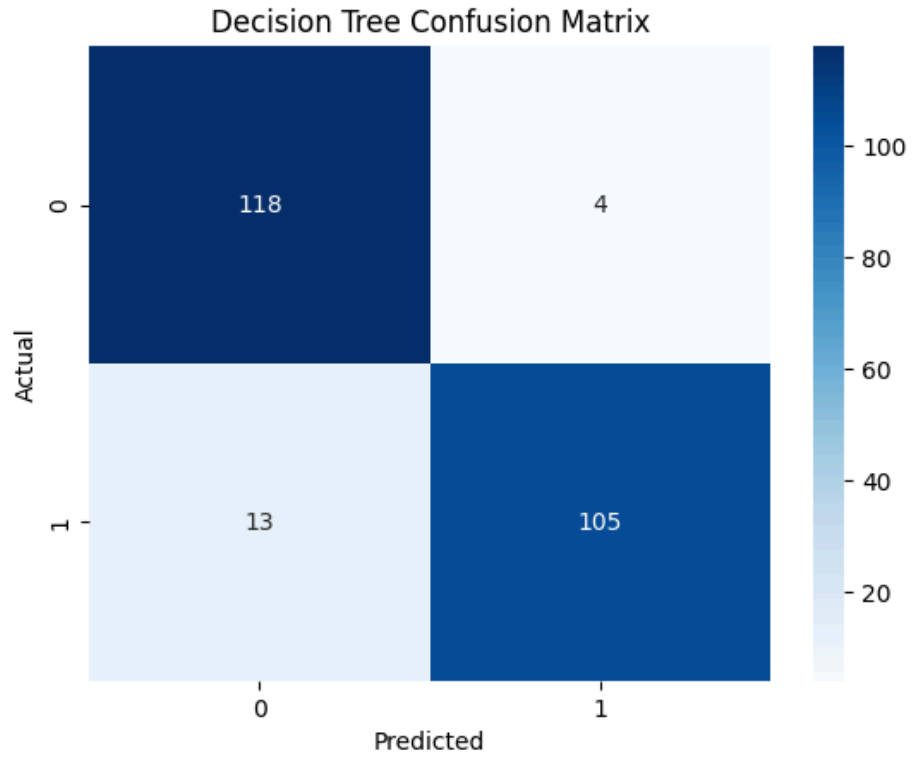

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

```
dt_model=DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train,Y_train)
dt_y_pred=dt_model.predict(X_test)
dt_acc=accuracy_score(Y_test,dt_y_pred)
```

```
print("Decision Tree Accuracy")
print(dt_acc)
```

```
Decision Tree Accuracy
0.9291666666666667
```

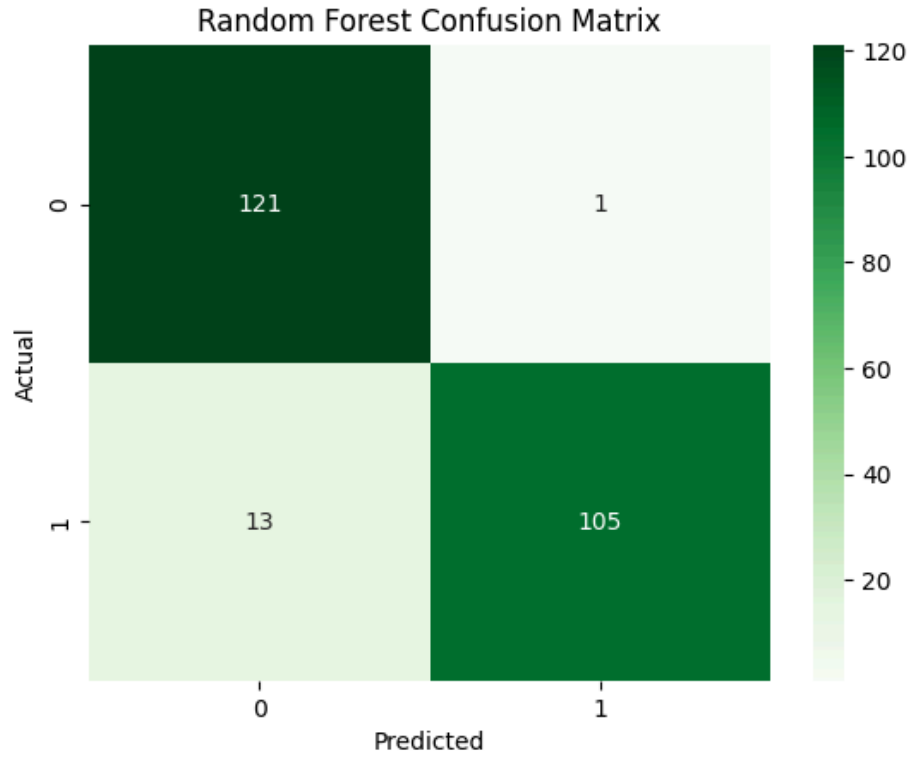
```
dt_cm = confusion_matrix(Y_test, dt_y_pred)
sns.heatmap(dt_cm, annot=True, fmt='d', cmap='Blues')
plt.title('Decision Tree Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```




```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, Y_train)
rf_y_pred = rf_model.predict(X_test)
rf_acc = accuracy_score(Y_test, rf_y_pred)
print(f"Random Forest Accuracy: {rf_acc * 100:.2f}%")
```

```
Random Forest Accuracy: 94.17%
```

```
rf_cm = confusion_matrix(Y_test, rf_y_pred)
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Greens')
plt.title('Random Forest Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```




```
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier

# Initialize and train KNN model to get Y_pred and accuracy
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, Y_train)
Y_pred = knn.predict(X_test)
accuracy = accuracy_score(Y_test, Y_pred)

print("--- MODEL COMPARISON ACCURACY ---")
print(f"1. KNN Accuracy : {accuracy * 100:.2f}%")
print(f"2. Decision Tree Accuracy : {dt_acc * 100:.2f}%")
print(f"3. Random Forest Accuracy : {rf_acc * 100:.2f}%")
```

```
--- MODEL COMPARISON ACCURACY ---
1. KNN Accuracy : 83.75%
2. Decision Tree Accuracy : 92.92%
3. Random Forest Accuracy : 94.17%
```

```
from sklearn.metrics import accuracy_score

print("--- Overfitting Check ---")

knn_train_acc = accuracy_score(Y_train, knn.predict(X_train)) * 100
knn_test_acc = accuracy_score(Y_test, knn.predict(X_test)) * 100
print(f"\n[KNN] Training: {knn_train_acc:.2f}%  Testing: {knn_test_acc:.2f}%")


dt_train_acc = accuracy_score(Y_train, dt_model.predict(X_train)) * 100
dt_test_acc = accuracy_score(Y_test, dt_model.predict(X_test)) * 100
print(f"[Decision Tree] Training: {dt_train_acc:.2f}%  Testing: {dt_test_acc:.2f}%")

rf_train_acc = accuracy_score(Y_train, rf_model.predict(X_train)) * 100
rf_test_acc = accuracy_score(Y_test, rf_model.predict(X_test)) * 100
```

```
print(f"[Random Forest] Training: {rf_train_acc:.2f}%  Testing: {rf_test_acc:.2f}%")
```

```
--- Overfitting Check ---

[KNN] Training: 89.17%  Testing: 83.75%
[Decision Tree] Training: 100.00%  Testing: 92.92%
[Random Forest] Training: 100.00%  Testing: 94.17%
```

```
ib.pyplot as plt
as sns
trics import confusion_matrix, accuracy_score

t accuracies within this cell to ensure they are defined.
knn, dt_model, rf_model, X_test, Y_test are defined in previous cells and are in scope.
accuracy_score(Y_test, knn.predict(X_test)) * 100
ccuracy_score(Y_test, dt_model.predict(X_test)) * 100
ccuracy_score(Y_test, rf_model.predict(X_test)) * 100

tyle="whitegrid")
ize=(6, 4))
, 'Decision Tree', 'Random Forest']
 = [knn_test_acc, dt_test_acc, rf_test_acc]

odels, y=accuracy_scores, palette='Blues_d')
)
uracy (%)')
odel Accuracy Comparison')


nerate(accuracy_scores):
 v + 1, f"{v:.1f}%", ha='center', fontweight='bold')



ize=(6, 4))

=y, palette='pastel')

s=[0, 1], labels=['Low Risk', 'High Risk'])
tal Health Risk Categories')
oer of Students')
istribution of Mental Health Risk Levels')


 np

ize=(5, 4))

el.predict(X_test)

= np.unique(y_pred, return_counts=True)


['Predicted Low', 'Predicted High'],
='%1.1f%%',
['#66b3ff', '#ff9999'],
gle=90)

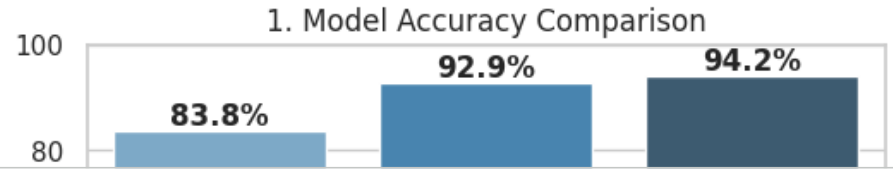
redicted Risk Levels (Pie Chart)')
```

```
)
```

```
/tmp/ipykernel_3165/461035191.py:16: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.barplot(x=models, y=accuracy_scores, palette='Blues_d')
```




```
print("="*60)
print("1. KNN CLASSIFICATION REPORT")
print("="*60)
knn_pred = knn.predict(X_test)
target_names = ['Low Risk', 'High Risk']
print(classification_report(Y_test, knn_pred, target_names=target_names, labels=[0, 1]))
```

```
============================================================
1. KNN CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

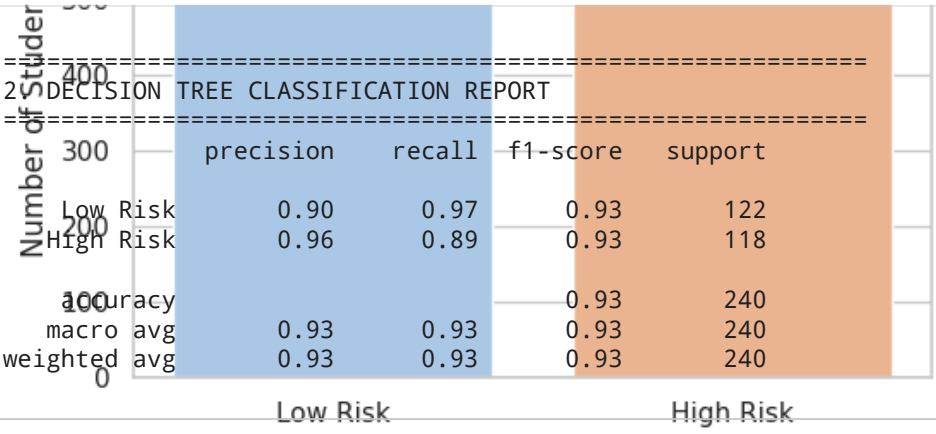
    Low Risk       0.80      0.90      0.85       122
   High Risk       0.88      0.77      0.82       118

    accuracy                           0.84       240
   macro avg       0.84      0.84      0.84       240
weighted avg       0.84      0.84      0.84       240

/tmp/ipykernel_3165/461035191.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.countplot(x=y, palette='pastel')
```



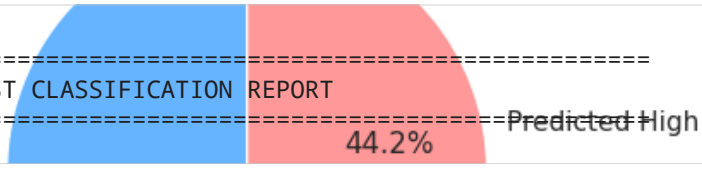

```
print("\n" + "="*60)
print("2. DECISION TREE CLASSIFICATION REPORT")
print("="*60)
dt_pred = dt_model.predict(X_test)
print(classification_report(Y_test, dt_pred, target_names=target_names, labels=[0, 1]))
```

```
============================================================
2. DECISION TREE CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

    Low Risk       0.90      0.97      0.93       122
   High Risk       0.96      0.89      0.93       118

    accuracy                           0.93       240
   macro avg       0.93      0.93      0.93       240
weighted avg       0.93      0.93      0.93       240
```

```
print("\n" + "="*60)
print("3. RANDOM FOREST CLASSIFICATION REPORT")
print("="*60)
rf_pred = rf_model.predict(X_test)
print(classification_report(Y_test, rf_pred, target_names=target_names, labels=[0, 1]))
```




```
============================================================
3. RANDOM FOREST CLASSIFICATION REPORT
============================================================
```